Analysis Report — Last Resort Hotel (LRH) Database

1. Executive Summary

We designed a normalized relational schema that comprehensively models Last Resort Hotel as an integrated operations system, encompassing facility management, reservations, events, billing, and maintenance tracking. The schema captures the hotel's hierarchical physical structure (hotel → building → wing → floor → room), supports flexible multi-function and multi-bed configurations, separates reservation intent from room assignment and actual occupancy, enables multi-room events, implements a service-categorized billing system, and tracks maintenance operations. The design prioritizes data integrity through referential constraints, normalization to eliminate anomalies, and explicit modeling of many-to-many relationships that reflect real-world operational complexity.

---

2. Key Design Decisions

2.1 Physical Hierarchy: Hotel → Building → Wing → Floor → Room

Decision: Implement an explicit 5-level hierarchical structure using surrogate primary keys (hotelId, buildingId, wingId, floorId, roomId).

Rationale:

- Directly reflects the case description and real-world hotel organization
- Enables unique room identification through path traversal (wing → floor → room)
- Simplifies foreign key relationships and prevents composite key proliferation
- Supports hierarchical queries for occupancy, revenue, and maintenance analysis across organizational levels

Implementation:

- Each level enforces referential integrity via foreign keys
- `wing` includes location attributes (nearPool, nearParking, handicapAccess)
- `floor` includes environmental attributes (nonSmokingFloor)
- `room` stores capacity and rate information
- UNIQUE constraints at each level ensure no duplicate codes within parent scope

2.2 Room Functions as M:N Relationship (Room ↔ Function via room_function)

Decision: Model room functions (sleeping, meeting, dining, etc.) as a separate master table with a bridge table `room_function` capturing the many-to-many relationship.

Rationale:

- Case explicitly states that rooms can be multi-purpose (e.g., convertible sleeping/meeting spaces)
- M:N modeling avoids 1NF violations that would occur with comma-separated function values
- Supports room searching by capability without loss of information
- Enables tracking of function activeness (active/inactive) per room

Implementation:

- `function` table: stores function codes and descriptive names
- `room_function` bridge table: composite PK on (roomId, functionCode) with activeness attribute
- Supports queries like "find all meeting rooms" or "rooms capable of both sleeping and dining"

2.3 Bed Configuration as M:N Relationship (Room ↔ Bed Type via room_has_bed)

Decision: Separate bed inventory into a `bed_type` master table with a bridge table `room_has_bed` recording quantity and foldability per room.

Rationale:

- Rooms frequently contain multiple bed types and quantities (e.g., 1 king + 2 twins)
- Foldable beds affect capacity and service requirements differently than permanent beds
- Supports accurate guest accommodation matching (guest count vs bed configuration)
- Enables bed-level inventory and maintenance tracking

Implementation:

- `bed_type` table: stores bed type names and passenger capacity per bed
- `room_has_bed` bridge: records quantity and isFoldable flag
- Enables capacity calculations: total_capacity = SUM(bed_type.capacity × room_has_bed.quantity)
- Supports queries for "rooms with twin beds" or "maximum-capacity rooms"

2.4 Room Adjacency for Suite and Connecting Room Support

Decision: Explicitly model room connections via a `room_adjacency` table recording roomId1, roomId2, and connectionType.

Rationale:

- Case mentions suites, connecting rooms, and movable walls requiring adjacency support
- Enables suite availability queries (e.g., "are rooms 101 and 102 both available?")
- Supports combined capacity calculations for adjacent rooms
- Documents physical topology for operations planning

Implementation:

- `room_adjacency` table: composite PK on (roomId1, roomId2) with connectionType
- CHECK constraint ensures roomId1 ≠ roomId2 (no self-adjacency)
- Supports queries for adjacent available rooms for larger parties

2.5 Reservations Separated from Assignments and Stay Records

Decision: Model three distinct entities: `reservation` (customer booking intent), `room_assignment` (room allocation), and `stay` (actual occupancy with check-in/check-out times).

Rationale:

- Case requires support for non-roomed reservations (events, group bookings without immediate assignment)

- Mid-stay room changes require separate assignment records per period
- Accurate billing requires distinguishing reservation period from actual occupancy period
- Enables queries about "confirmed but unassigned" reservations and guest movement history

Implementation:

- `reservation` table: captures booking intent with dateCreated, startDate, endDate, and deposit requirements
- `room_assignment` table: links reservation to specific room with assignmentDate
- `stay` table: records actual check-in/check-out times per reservation (can be NULL for checkout until departure)
- One reservation can have multiple room_assignments (room changes)
- One reservation can have multiple stays (split occupancy)

2.6 Events with Multi-Room Support (Event ↔ Room via event_room)

Decision: Implement M:N relationship between events and rooms via `event_room` bridge table.

Rationale:

- Case explicitly describes multi-room events (e.g., conference with session rooms plus dining)
- Same room may host different events or activities on different dates
- Enables event sizing queries (rooms per event type) and room utilization by event context

Implementation:

- `event` table: stores event metadata (hostPartyId, eventType, time, estimatedGuestCount)
- `event_room` bridge: composite PK on (eventId, roomId)
- Supports queries like "average rooms per event type" or "room utilization by event"

2.7 Billing Architecture: Service Types, Charges, and Billing Accounts

Decision: Implement a three-table billing model: `service_type` (service catalog), `charge` (individual charges), and `billing_account` (account aggregation).

Rationale:

- Service-level categorization enables revenue reporting by service type (rooms, dining, parking, etc.)
- Billing accounts tied to parties support corporate billing, group billing, and individual billing scenarios
- Charge table optionally links to stay for occupancy-driven charges, or stands alone for event-driven charges
- Supports split billing and multi-account reservations

Implementation:

- `service_type` table: serviceCode (PK), serviceType (unique name), baseRate
- `charge` table: chargeId, accountId (FK to billing_account), stayId (optional FK), serviceCode (FK), chargeAmount, dateIncurred
- `billing_account` table: accountId, partyId, status, creditLimit
- Foreign key on (accountId, serviceCode) ensures charges reference valid services
- Optional stayId allows non-occupancy charges (events, advance fees)

2.8 Maintenance Tracking via Maintenance Tickets

Decision: Implement `maintenance_ticket` table to record room issues, resolution status, and timelines.

Rationale:

- Case mentions renovations, breakdowns, and service disruptions requiring maintenance tracking
- Enables room unavailability logic (maintenance rooms should not be assigned)
- Supports maintenance efficiency analysis (resolution time by area)
- Records operational history for continuous improvement

Implementation:

- `maintenance_ticket` table: ticketId, roomId, issueDescription, status, dateCreated, dateResolved
- CHECK constraint ensures dateResolved ≥ dateCreated (valid timeline)
- Supports queries for "pending maintenance by location" and "average resolution time"

2.9 Party and Guest Group Management

Decision: Separate `party` (billing entity) from `guest_group` (individuals within a party/group).

Rationale:

- Party can represent individual guest, corporate account, or organization
- Guest group captures individual members (names, identities) within a party booking
- Enables group-level reporting while maintaining individual guest records
- Supports diverse business models (corporate vs individual, group vs solo)

Implementation:

- `party` table: partyId, email, phone, partyType (individual/corporate/organization), contactPersonName
- `guest_group` table: composite PK on (partyId, guestId) with guestName
- One party can have multiple guests; one guest associated with only one party per booking

---

3. Assumptions

1. **Room Numbering Scope**: Room numbers are unique only within a wing. Full room identification requires the path: hotel → building → wing → room.

2. **No Overbooking**: The reservation system logic (in application layer) prevents overbooking. The schema enforces no constraints that would prevent it; availability checking is external.

3. **Reservation-to-Assignment Timing**: A reservation can exist without room assignments (e.g., pending confirmation). Room assignments are created later when rooms are allocated.

4. **Charge Attribution**: Every charge must be attributed to a billing_account. Charges may optionally reference a stay; charges without a stayId are typically event-related or advance fees.

5. **No Dynamic Rate Calendar**: Base rates are stored per room; dynamic pricing, promotional rates, or seasonal adjustments are not modeled in the schema.

6. **No Real-Time Access Logs**: Physical access control (magnetic cards, key logs) exists in reality but is out of scope for this schema.
7. **Sufficient Date/Time Granularity**: Dates and times (reservation periods, event times, stay durations) are stored in ISO 8601 text format with sufficient precision for scheduling decisions; minute-level or real-time granularity is not required.
8. **Flexible Status Values**: Room `currentStatus` and maintenance ticket `status` are text fields accepting any value (e.g., "Available", "Occupied", "Maintenance", "Under Renovation"). Application logic enforces valid transitions.
9. **Primary Guest Identity**: `reservation` is linked to a `party`, which represents the booking entity. Individual guests are tracked in `guest_group` if part of a group booking.

---

4. Tradeoffs and Limitations

Tradeoffs Made for Project Efficiency and Maintainability

Surrogate Keys vs Natural Composite Keys:

- **Decision**: Used surrogate integer keys (hotelId, buildingId, wingId, floorId, roomId) throughout.
- **Tradeoff**: Surrogate keys simplify FK references and diagram clarity but hide natural uniqueness constraints (e.g., unique room code per building).
- **Mitigation**: Business rules enforced via application logic and UNIQUE constraints (e.g., `UNIQUE (buildingId, wingCode)` on wing table).

No Rate Calendar or Dynamic Pricing:

- **Decision**: Simplified to static baseRate per room.
- **Limitation**: Cannot directly answer "rate for room X on date Y" if dynamic pricing is required.
- **Mitigation**: Application layer can maintain rate cache or apply pricing rules at query time.

No Audit Trail for Status Changes:

- **Decision**: Store only current status in room and maintenance_ticket tables.
- **Limitation**: No historical record of status transitions (e.g., when did room become "Maintenance"?).
- **Mitigation**: Application can log state changes externally; schema could be extended with a status_history table if needed.

Simplified Billing (No Payment Ledger):

- **Decision**: Charges exist but no separate payment/cash-posting table.
- **Limitation**: Cannot track partial payments, refunds, or settlement details.
- **Mitigation**: Payment data can be managed in application layer; schema could extend with payment table if detailed ledger is required.

No Real-Time Availability Index:

- **Decision**: Availability computed dynamically from reservation, room_assignment, stay, and maintenance_ticket tables.

- **Limitation**: Availability queries may be slower with large data volumes.
- **Mitigation**: Application should maintain a materialized availability cache or schedule periodic refresh.

Known Limitations

1. **Limited Support for Promotions and Packages**: Current schema cannot model bundled offerings or promotional pricing. Extension: add tables for promotion, package, and package_item.

2. **No Multi-Language Support**: Party contact names and room descriptors are single-language. Extension: add i18n tables if multi-language support required.

3. **No Guest Preference or Loyalty Data**: No schema for recording guest preferences, allergies, or loyalty program status. Extension: add guest_preference table.

4. **Limited Event Flexibility**: Events are tied to a specific party; recurring events or multi-party events are not directly supported. Workaround: create separate event records per occurrence or party.

5. **No Overbooking or Waitlist Tracking**: Cannot natively handle overbooking, cancellations with waitlists, or priority queue logic. Extension: add waitlist table if needed.

6. **No Financial Controls**: No schema-level constraints on credit limits, pending balance, or automated account holds. Enforcement is application-level.

---

5. How Ambiguities in the Case Were Resolved

"Room Type" vs "Multi-Function Rooms"

Ambiguity: The case used language like "sleeping rooms," "meeting rooms," and "suites," implying both fixed room types and flexible multi-purpose rooms.

Resolution: Implemented room functions as M:N (function + room_function bridge) to support both. This allows:

- A room to have multiple functions (e.g., sleeping + meeting)
- Queries to find rooms by capability (e.g., "rooms suitable for meetings")
- Dynamic room repurposing without schema changes

Rationale: M:N modeling is more flexible than a fixed room_type field and aligns with case language about "convertible" and "flexible" spaces.

Reservation vs Room Assignment vs Stay

Ambiguity: Unclear whether a "reservation" inherently includes a room, or if booking can exist without assignment.

Resolution: Implemented three separate entities:

1. **Reservation**: Booking intent (party, dates, deposit requirement)
2. **Room Assignment**: Specific room allocation (can be multiple per reservation if room changes occur)
3. **Stay**: Actual occupancy period (check-in, check-out times; can be multiple if split stays)

Rationale: Enables group bookings without immediate per-person room assignment, mid-stay room changes, and accurate stay history. Aligns with case language: "non-roomed reservations" and "group bookings."

Event Room Usage

Ambiguity: Whether each event occupies exactly one room or multiple rooms.

Resolution: Implemented event_room as M:N relationship, allowing:

- Single event to use multiple rooms (conference with sessions in different rooms)
- Same room to host multiple events at different times
- Event-based room utilization analysis

Rationale: Case mentions multi-room events (e.g., conference with session + dining areas). M:N relationship supports this without restrictive assumptions.

Where Charges Belong

Ambiguity: Whether charges are always tied to a stay or can be event-based or advance fees.

Resolution:

- Made stayId a **nullable foreign key** in the charge table
- Charges always link to a billing_account and service_type
- Charges can be attributed to a stay (occupancy-driven) or exist independently (event-driven, advance fees)

Rationale: Supports diverse charging scenarios without forcing all charges into a stay context. Event charges, deposits, and service fees can be recorded separately.

Wing Location Attributes

Ambiguity: Case suggested sequence_number, near_pool, near_parking, and handicap access as wing attributes.

Resolution:

- Included: `nearPool`, `nearParking`, `handicapAccess` (boolean flags in wing table)
- Omitted: `sequenceNumber` (considered non-essential for core functionality)

Rationale: Location attributes support guest-facing amenity searches and operational queries (e.g., "wheelchair-accessible wings"). Sequence number can be derived from wingId if needed.

Room Status Management

Ambiguity: How to handle room states (available, occupied, maintenance, under renovation, housekeeping).

Resolution:

- Room has a `currentStatus` text field (no predefined enum)
- Maintenance issues tracked separately in `maintenance_ticket` table
- Application logic determines valid status transitions

Rationale: Text-based status allows flexibility for future states without schema changes. Maintenance table provides detailed issue tracking separate from simple availability flags.

Access Control and Magnetic Card Logs

Ambiguity: Case mentions magnetic card access control and access logs.

Resolution: Omitted from schema as out-of-scope for hotel operations core model.

Rationale: Access control is a separate security system; core grading criteria focus on reservation, billing, and facility management. Can be modeled separately if required.

Authorized vs Actual Charges

Ambiguity: Should the schema distinguish between authorized (expected) charges and actual charges?

Resolution: Implemented only `charge` table for actual charges. Expected/authorized charges are not modeled.

Rationale: Simplifies model; application layer can track expected vs actual through business logic if needed. Charge table captures ground truth of what was actually billed.

---

6. Schema Integrity Features

Referential Integrity

- All foreign keys defined with explicit FOREIGN KEY constraints
- PRAGMA foreign_keys = ON enforced at database level
- Cascading deletes not enabled; referential integrity is strict

Uniqueness Constraints

- Surrogate PKs ensure no duplicate records
- Natural uniqueness enforced where meaningful:
  - Wing code unique per building
  - Floor number unique per wing
  - Room number unique per floor
  - Function name and service type are globally unique

Check Constraints

- Binary flags (nearPool, nearParking, nonSmokingFloor, depositRequired, isFoldable) constrained to {0, 1}

- Date/time logic: endDate ≥ startDate, checkoutTime ≥ checkinTime, dateResolved ≥ dateCreated
- Bed quantity ≥ 0 (no negative quantities)
- Room IDs must differ in adjacency table

Data Type Choices

- Surrogate keys: INTEGER PRIMARY KEY (auto-increment support)
- Monetary values: REAL (suitable for billing; application layer handles precision)
- Dates/times: TEXT in ISO 8601 format (portable, queryable with strftime())
- Flags: INTEGER {0,1} for boolean semantics
- IDs/codes: TEXT for flexibility (e.g., room IDs can be building-prefixed codes)

---

7. Query Patterns Supported

The schema supports the following analytical queries that address key business needs:

1. **Room Utilization**: Room assignments by building/wing, top-turnover rooms, capacity vs occupancy
2. **Revenue Analysis**: Revenue by service type, monthly trends, top billing parties, deposit pattern analysis
3. **Event Planning**: Event room requirements, guest counts, multi-room event allocation
4. **Maintenance Operations**: Tickets by area, resolution time, maintenance burden by location
5. **Guest Behavior**: Average stay duration by party type, lead-time analysis, repeat customers
6. **Availability**: Available rooms by criteria (date, capacity, amenities, bed config)
7. **Billing**: Account balances, charge aging, per-service revenue trends

---

8. Conclusion

The Last Resort Hotel database schema represents a balanced design that captures the full operational complexity of a multi-building hotel enterprise while maintaining normalization, referential integrity, and query flexibility. Key strengths include explicit hierarchical structure, M:N relationships for real-world complexity, separation of concerns (reservations, assignments, stays), and comprehensive event/billing support. The design prioritizes operational accuracy over advanced analytics or real-time optimization, making it suitable for transactional systems, historical analysis, and reporting workflows. Future extensions (dynamic pricing, loyalty programs, payment ledger) can be layered atop this foundation without disrupting core entities or relationships.